

Reviewer Pack — Coxeter Group Complexity of SAT Proof Trees via Reflection P...

Entry 86ccf8b95619 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 04:32:40 UTC

Coxeter Group Complexity of SAT Proof Trees via Reflection Posets

Entry ID: 86ccf8b95619 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 04:32:40 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory (specifically, Coxeter groups and reflection posets) **Field B** (complexity object): Resolution Proof Trees in Complexity Theory

Statement:

For every Boolean satisfiability problem with n variables and m clauses, the resolution proof width of the problem is upper-bounded by $O(n^{1/3} * m^{2/3}) + \log(m)$, where the complexity of the associated reflection poset is measured as the minimum number of generators needed to generate all reflections in the Coxeter group representing the clause set.

Rationale (proposer's reasoning):

Reflection posets provide a combinatorial structure that encodes the symmetries of Boolean functions, and their complexity can reveal underlying patterns in proof structures. The conjecture leverages this to propose a new way of measuring proof width, which could lead to more efficient algorithms or hardness results for resolution.

Taxonomy category: Coxeter_group_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ac2f6a4411d19b8a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all instances with $n \leq 40$ and $m \leq 200$, the resolution proof width is less than or equal to $O(n^{1/3} * m^{2/3}) + \log(m)$, and the complexity of the reflection poset is upper-bounded by a constant factor of this bound. The conjecture is falsified if any instance with $n \leq 40$ and $m \leq 200$ violates this condition.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter groups" AND "resolution proof trees" - "reflection posets" AND "SAT proof complexity" - "Boolean satisfiability problem" AND "Coxeter group generators"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=9.2s

5.1 Generated Python source

```
import random
import math

def generate_instance(n: int, m: int) -> list:
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause_size = random.randint(1, n)
        clause = sorted(random.sample(variables, clause_size))
        clauses.append(clause)
    return clauses

def calculate_coxeter_group_complexity(clauses: list) -> int:
    # Placeholder function to compute the complexity of the reflection poset
    # This is a dummy implementation and should be replaced with actual logic
    return len(clauses)

def calculate_resolution_proof_width(n: int, m: int) -> float:
    # Placeholder function to compute the resolution proof width
    # This is a dummy implementation and should be replaced with actual logic
    return n * m

def run_trial(seed: int) -> dict:
    random.seed(seed)

    metric_name = "resolution_proof_width"
    instances_tested = 0
    total_metric_value = 0.0
    n_max = 0
    conjecture_holds = True
    counterexample = ""
```

```

for n in [5, 10, 15, 20, 30, 40]:
    for m in range(1, min(200, int(n**2/2) + 1)):
        instances_tested += 1
        clauses = generate_instance(n, m)
        complexity = calculate_coxeter_group_complexity(clauses)
        width = calculate_resolution_proof_width(n, m)

        n_max = max(n_max, n)
        total_metric_value += width

        upper_bound = n**(1/3) * m**(2/3) + math.log(m)
        if width > upper_bound:
            conjecture_holds = False
            counterexample = f"n={n}, m={m}: width={width} > {upper_bound}"

mean_metric_value = total_metric_value / instances_tested
result = {
    "metric_name": metric_name,
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{r['counterexample']}'\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

ure_holds': False, 'counterexample': 'n=40, m=199: width=7960 > 121.86381970652506'

3. The full LLM transcripts are in `research/audit/86ccf8b95619.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/86ccf8b95619.tar.gz` (if generated)